

Sprecherskript — Kevin Kunkel

Recurrent Graph Neural Networks: Logical Characterizations via Modal Logic

Seminar GNNs · Universität Leipzig · SoSe 2026

Teil 1: Section 2.2 — Logics

5 min

Teil 2: Section 4 — MSO

20 min

Teil 3: Fazit (gemeinsam)

5 min

Teil 1: Section 2.2 — Logics

Übergang von Thomas nach etwa 5 Minuten GNN-Einführung

FOLIE: Graded Modal Logic (GML)

 ca. 2 min

Hinweis: Erster Folienwechsel zu meinem Teil — kurz einleiten, dann direkt einsteigen.

Danke, Thomas. Ich übernehme jetzt mit den Logiken — also dem formalen Rahmen, den wir brauchen, um GNNs präzise zu charakterisieren.

Die erste Logik ist die **Graded Modal Logic**, kurz **GML**. Man kann GML als Aussagenlogik mit Zählmodalitäten beschreiben. Die Syntax ist vertraut — Top, Atome, Negation, Konjunktion — aber es kommt ein neuer Operator dazu: der **Diamant mit Zähler k**. Dieser Ausdruck $\Diamond_{\geq k} \varphi$ liest sich als: „Es gibt mindestens *k* direkte Nachfolger, die *phi* erfüllen.“

Das ist genau die Art von lokalem Zählen, die ein GNN in einer Runde machen kann. Schauen wir auf das Beispiel: $\Diamond_{\geq 1} p$ sagt einfach „es gibt einen Nachbarn mit Label *p*“. Anspruchsvoller: $\Diamond_{\geq 3} (q \wedge \Diamond_{\geq 2} r)$ sagt „mindestens 3 Nachbarn haben Eigenschaft *q*, und jeder davon hat seinerseits mindestens 2 Nachbarn mit *r*“. Man sieht, wie sich diese Formeln schachteln lassen — genau wie Runden im Message Passing.

Warum ist GML überhaupt relevant? Es gibt ein klassisches Ergebnis von Barceló et al. (2020): GNNs mit **fester** Rundenzahl sind äquivalent zu GML — aber nur relativ zu FO-definierbaren Eigenschaften. Und das ist der entscheidende Punkt: die Einschränkung „feste Runden“ passt zu standard GNNs, nicht zu recurrent GNNs. Für rekurrente Netze brauchen wir mehr Ausdrucksstärke — und dafür die nächste Logik.

FOLIE: Graded Modal Substitution Calculus (GMSC)

 ca. 1:30 min

GMSC — das **Graded Modal Substitution Calculus** — erweitert GML um **rekursive Regeln**.

Ein GMSC-Programm Lambda besteht aus zwei Teilen, die ihr auf der Folie seht. Die **Basisregel** $X(0) :- \varphi$ gibt uns eine GML-Formel für den Anfangszustand — Runde 0. Die **Iterationsregel** $X :- \psi$ beschreibt, wie wir iterativ weitermachen — wobei ψ selbst wieder X verwenden darf.

Die n -te Entfaltung X^n entsteht durch sukzessives Einsetzen: wir nehmen ψ und ersetzen X durch X^{n-1} , beginnend bei $X^0 = \varphi$. Lambda akzeptiert einen Knoten v im Graphen G , wenn es **irgendein** n gibt, sodass v die Formel X^n erfüllt.

Das Standardbeispiel illustriert das perfekt: **Erreichbarkeit von Label p** . Basisfall: die Formel p selbst — der Knoten hat p . Iterationsregel: ein Schritt weiter — der Diamant angewendet auf X . Die n -te Entfaltung ist dann $\diamond \dots \diamond p$ mit n Diamanten — also „ p ist in genau n Schritten erreichbar“. Da Lambda für **irgendein** n akzeptiert, erfasst es genau die Eigenschaft „ p ist von diesem Knoten aus erreichbar“. Und das ist unbeschränkt — egal wie groß der Graph ist.

Das entspricht exakt dem, was ein Recurrent GNN tut: es läuft, bis es die Antwort gefunden hat.

FOLIE: ω -GML und die Logikhierarchie

 ca. 1:30 min

Hinweis: Folie zeigt die drei Logiken und ihre Hierarchie — kurz auf alle drei eingehen, dann sauber zu Thomas überleiten.

Die dritte Logik ist ω -GML. Sie nimmt GML und erlaubt zusätzlich **unendliche Disjunktionen** über abzählbare Mengen von GML-Formeln.

Warum brauchen wir das? Weil $\text{GNN}[R]$ — also GNNs mit reellen Zahlen — beliebig viele verschiedene Werte unterscheiden kann. Konkret kann ein $\text{GNN}[R]$ für jede beliebige Menge $U \subseteq \mathbb{N}$ prüfen, ob die Anzahl der Nachbarn in U liegt — auch für **undecidierbare** Mengen U . Um diese Ausdrucksstärke logisch zu erfassen, reichen endliche Disjunktionen nicht — wir brauchen ω -GML mit ihren unendlichen Disjunktionen.

Das ergibt die Hierarchie, die ihr auf der Folie seht: $\text{GML} \subset \text{GMSC} \subset \omega\text{-GML}$ Jede dieser Inklusionen ist **echt** — die Logiken werden also tatsächlich stärker.

Zur Einordnung: Auf Strings ist FO äquivalent zu sternfreien regulären Sprachen, MSO zu allen regulären Sprachen. Auf Graphen kann FO lokale Dinge sagen wie „jeder Knoten hat einen Nachbarn“, MSO kann globale strukturelle Eigenschaften erfassen — Bipartitheit, Erreichbarkeit, und mehr.

Diese drei Logiken sind die formalen Werkzeuge. Thomas zeigt euch jetzt, wie sie über Automaten mit den GNNs verbunden werden.

Hinweis: Übergabe an Thomas für Section 3 — ca. 20 Minuten.

Teil 2: Section 4 — GNNs über MSO-Eigenschaften

Rückübernahme nach Thomas' Section 3 (ca. 20 Minuten)

FOLIE: Was ist MSO?

 ca. 3 min

Hinweis: Direkt nach Thomas' Haupttheoreme übernehmen — kurze Einleitung und dann motivieren, warum MSO die richtige Einschränkung ist.

Vielen Dank, Thomas. Ich übernehme jetzt den letzten Teil — Section 4: GNNs über MSO-Eigenschaften.

Wir haben gerade gesehen, dass $\text{GNN}[R]$ und $\omega\text{-GML}$ äquivalent sind — und dass $\omega\text{-GML}$ sogar unentscheidbare Eigenschaften ausdrücken kann. Das ist theoretisch faszinierend, aber praktisch stellt sich die Frage: *Wer will schon unentscheidbare Grapheigenschaften lernen?* In der Praxis interessieren uns Dinge wie Zusammenhang, Bipartitheit, Färbbarkeit — Eigenschaften, die wir auch verstehen und verifizieren können.

Genau das erfasst **Monadische Zweistufenlogik**, kurz **MSO**. MSO erweitert Erstlogik durch **Mengenquantifikation**. In FO dürfen wir über Elemente quantifizieren — „es gibt ein x “, „für alle x “. In MSO dürfen wir **zusätzlich** über Mengen von Elementen quantifizieren — „es gibt eine Menge X “, „für alle Mengen X “. Dieser Schritt klingt klein, öffnet aber enorme Ausdrucksmacht.

Schauen wir auf die Folie. Auf Graphen: FO kann sagen „jeder Knoten hat einen Nachbarn“, — eine lokale, knotenweise Eigenschaft. MSO kann sagen „der Graph ist bipartit“ — dafür quantifizieren wir existenziell über eine Menge X von Knoten und sagen: X und sein Komplement sind unabhängige Mengen. MSO kann auch Erreichbarkeit ausdrücken, Hamiltonizität, k -Färbbarkeit — viele fundamentale Graphprobleme.

Das String-Beispiel auf der Folie zeigt, wie klassisch das ist: FO entspricht genau den sternfreien regulären Sprachen, MSO entspricht **allen** regulären Sprachen. Das ist ein sauberes, wohlbekanntes Ergebnis — und MSO auf Graphen ist das natürliche Analogon.

MSO ist in gewissem Sinne die **natürliche Logik für Graphen** — sie erfasst fast alles, was man in der Praxis wirklich braucht. Und jetzt stellen wir die entscheidende Frage: Was passiert, wenn wir GNNs auf genau diese praktisch relevanten Eigenschaften einschränken?

FOLIE: MSO — Was MSO ausdrücken kann

 ca. 2 min

Konkret: Was liegt in MSO? Die Folie zeigt eine Auswahl.

Zusammenhang, k -Färbbarkeit, Hamiltonizität, Bipartitheit, Planarität für festes Geschlecht, Erreichbarkeit zwischen zwei markierten Knoten — das sind alles MSO-ausdrück-

bare Eigenschaften. Das sind fundamentale Probleme der Graphentheorie, die in maschinellem Lernen auf Graphen ständig auftauchen.

Aber hier eine wichtige Klarstellung: MSO ist **nicht** die absolute Grenze dessen, was GNNs können. Thomas hat in Section 3 gezeigt: $\text{GNN}[\text{R}]$ kann sogar **undecidierbare** Eigenschaften ausdrücken — zum Beispiel „der Grad ist eine Primzahl“, für beliebig große Grade. Das liegt weit jenseits von MSO.

MSO ist also eine **Einschränkung** — aber eine, die genau den praxisrelevanten Bereich abdeckt. Die Frage ist: Wenn wir uns auf MSO einschränken — verschwindet dann der Unterschied zwischen $\text{GNN}[\text{R}]$ und $\text{GNN}[\text{F}]$? Die Antwort ist ja — und das ist das überraschende Ergebnis dieses Abschnitts.

(kurze Pause)

FOLIE: MSO — Theorem 4.1 und Beweiszutaten

 ca. 7 min

Hinweis: Das ist die technisch dichteste Folie — ruhig und strukturiert vorgehen. Die zwei Beweiszutaten sind der Kern, beide erklären.

Kommen wir zum zentralen Resultat: **Theorem 4.1.**

Das Theorem besagt: Für jede MSO-ausdrückbare Eigenschaft $\mathcal{P}$ gilt — $\mathcal{P}$ ist in $\text{GNN}[\text{R}]$ ausdrückbar **genau dann, wenn** $\mathcal{P}$ in GMSC ausdrückbar ist. Kombiniert mit Theorem 3.2 von Thomas ergibt das: Über alle MSO-Eigenschaften sind $\text{GNN}[\text{F}]$, $\text{GNN}[\text{R}]$, und GMSC **alle äquivalent**.

Moment — das ist wirklich überraschend. Wir wissen aus Section 3: absolut gesehen ist $\text{GNN}[\text{R}]$ **stärker** als $\text{GNN}[\text{F}]$ und GMSC. $\text{GNN}[\text{R}]$ kann Dinge, die $\text{GNN}[\text{F}]$ und GMSC nicht können. Aber sobald wir uns auf MSO einschränken, **kollabiert** diese Hierarchie vollständig. Wie kann das sein?

Der Beweis hat **zwei Schlüsselzutaten**.

Erste Zutat: Das Janin-Walukiewicz-Theorem (Theorem 4.6 im Papier).

GMSC und ω -GML sind **modale** Logiken — sie sind invariant unter Baumentfaltung. Das bedeutet: Wenn zwei Knoten in verschiedenen Graphen dieselbe unendliche Baumentfaltung haben, können modale Logiken sie nicht unterscheiden. Das schränkt die Ausdrucksstärke ein.

Das Janin-Walukiewicz-Theorem verbindet nun MSO und modale Logik: Es charakterisiert MSO auf Bäumen durch **Paritätsbaumautomaten**, kurz PTAs. Und hier ist der entscheidende Schritt — wenn eine Eigenschaft $\mathcal{P}$ sowohl in MSO als auch in ω -GML liegt, dann gibt es einen PTA $\mathcal{A}$, sodass gilt: $(G, w) \in \mathcal{P}$ genau dann, wenn die Baumentfaltung $U(G, w)$ in der Sprache von $\mathcal{A}$ liegt.

Wir übersetzen also das Problem: statt direkt mit GNNs zu arbeiten, schauen wir uns an, ob der entfaltete Baum vom Automaten akzeptiert wird. Das ist eine Brücke – von der Grapheigenschaft zu einem Baumautomaten.

(kurze Pause)

Zweite Zutat: k-Präfix-Dekorationen (Lemma 4.7).

Das ist der technische Kern des Beweises. Für einen PTA $\mathcal{A}$ definiert man eine **Tiefe-k-Karte** μ : Sie ordnet jedem Knoten des Baums in Tiefe k eine Menge von PTA-Zuständen zu. Das ist ein **endliches Zertifikat** dafür, dass der Baum von $\mathcal{A}$ akzeptiert wird.

Das Lemma sagt dann: Ein Baum liegt in $L(\mathcal{A})$ – also wird vom Automaten akzeptiert – genau dann, wenn er eine k -Präfix-Dekoration für **irgendein** k hat.

Warum ist das so wichtig? Weil ein **GMSC-Programm** genau das überprüfen kann! Runde k des GMSC prüft, ob das Tiefen- k -Zertifikat existiert. Da GMSC für **irgendein** n akzeptiert, ist das exakt die Semantik, die wir brauchen.

Die Beweisidee in einem Satz: Die MSO-Eigenschaft wird zu einem PTA, der PTA gibt uns endliche Zertifikate, und die Zertifikate werden von einem GMSC-Programm überprüft. Damit ist jede MSO-Eigenschaft in $\text{GNN}[\mathbb{R}]$ auch in GMSC – und da $\text{GNN}[\mathbb{F}]$ äquivalent zu GMSC ist, auch in $\text{GNN}[\mathbb{F}]$.

FOLIE: Das MSO-Kollaps-Theorem

 ca. 5 min

Hinweis: Das ist das Hauptresultat – klarer Kontrast zwischen „absolut,“ und „relativ zu MSO“ herausarbeiten. Folie hat zwei Kästen dafür.

Theorem 4.3 – das MSO-Kollaps-Theorem – ist das Herzstück von Section 4.

Der Satz: Für jede MSO-ausdrückbare Eigenschaft $\mathcal{P}$ gilt: $\mathcal{P}$ ist durch $\text{GNN}[\mathbb{R}]$ ausdrückbar **genau dann, wenn** $\mathcal{P}$ durch $\text{GNN}[\mathbb{F}]$ ausdrückbar ist. Und beide werden außerdem durch GMSC erfasst. Das gilt auch für GNNs mit konstanter Iterationszahl.

Schauen wir uns die beiden Kästen auf der Folie an – sie zeigen den perfekten Kontrast.

Links, **absolut gesehen**: $\text{GNN}[\mathbb{R}]$ ist strikt mächtiger als $\text{GNN}[\mathbb{F}]$. Das haben wir in Section 3 gesehen. Der Unterschied liegt bei Eigenschaften wie „der Grad ist eine Primzahl,“ für beliebig großen Grad – $\text{GNN}[\mathbb{R}]$ kann das, $\text{GNN}[\mathbb{F}]$ nicht, weil Floats ab einem gewissen Punkt nicht mehr zählen können. Und es geht noch weiter: $\text{GNN}[\mathbb{R}]$ kann sogar **undecidierbare** Eigenschaften ausdrücken.

Rechts, **relativ zu MSO**: $\text{GNN}[\mathbb{R}]$ und $\text{GNN}[\mathbb{F}]$ sind äquivalent. Für alle praktisch relevanten Eigenschaften – alles, was in MSO liegt – sind Fließkommazahlen **genauso gut** wie reelle Zahlen.

Was bedeutet das intuitiv? Die Extramacht von $\text{GNN}[\mathbb{R}]$ liegt in einem Bereich, der für die Praxis vollständig irrelevant ist – bei Eigenschaften, die niemand in einem echten

ML-System lernen möchte. Sobald wir uns auf den praxisrelevanten Bereich beschränken, **kollabiert** die Hierarchie.

Das ist ein elegantes Ergebnis — und eines, das echte praktische Konsequenzen hat. Dazu komme ich auf der nächsten Folie.

FOLIE: Warum der Unterschied verschwindet — Intuition

 ca. 3 min

Hinweis: Diese Folie fasst die Beweisidee nochmal zusammen — ruhig und klar, keine neuen technischen Details.

Schauen wir nochmal auf die Beweisstruktur — jetzt als Übersicht.

Der Weg geht über **Paritätsbaumautomaten**. Das Janin-Walukiewicz-Theorem zeigt, dass MSO auf baumstrukturierten Graphen genau durch PTAs charakterisiert wird. Das gibt uns folgende Kette:

Eine MSO-Eigenschaft $\mathcal{P}$ wird in einen PTA $\mathcal{A}$ übersetzt. Aus $\mathcal{A}$ konstruiert man k -Präfix-Dekorationen — das sind endliche Zertifikate der Tiefe k . Und aus diesen Dekorationen baut man ein GMSC-Programm Λ , das in Runde k prüft, ob das Tiefen- k -Zertifikat existiert.

Der entscheidende Punkt — und das ist die Kernaussage des ganzen Abschnitts: Die Extra-macht von $\text{GNN}[\mathbb{R}]$ liegt **vollständig außerhalb von MSO**. Wenn eine Eigenschaft $\mathcal{P}$ in MSO ist und durch $\text{GNN}[\mathbb{R}]$ ausdrückbar ist, dann lässt sich immer ein GMSC-Programm — und damit ein $\text{GNN}[\mathbb{F}]$ — für $\mathcal{P}$ finden.

Im grünen Kasten auf der Folie steht die praktische Konsequenz, und die ist wirklich relevant: **Theoretische Analysen mit reellen Zahlen sind sicher**. Für MSO-Eigenschaften übertragen sich alle Ergebnisse auf Fließkommazahlen. Wenn ein GNN eine MSO-definierbare Eigenschaft nicht lernt — sagen wir, Bipartitheit oder Zusammenhang — dann liegt das Problem im Training oder in der Architektur. **Nicht** an der Float-Präzision. Das ist eine starke Aussage für alle, die GNNs theoretisch analysieren und die Ergebnisse auf die Praxis übertragen wollen.

(kurze Pause)

Teil 3: Fazit (gemeinsam mit Thomas)

Kevin beginnt das Fazit, Thomas ergänzt open questions nach Absprache

FOLIE: Results at a Glance

 ca. 2:30 min

Hinweis: Tabelle auf der Folie zeigt alle Ergebnisse — jede Zeile kurz erläutern, dann das Gesamtbild zusammenfassen.

Fassen wir alle Ergebnisse zusammen.

Die Tabelle auf der Folie zeigt zwei Perspektiven: oben die **absoluten** Ergebnisse, unten die Ergebnisse **relativ zu MSO**.

Absolut: GNN[F] ist äquivalent zu GMSC — das ist Theorem 3.2. GNN[R] ist äquivalent zu ω -GML — das ist Theorem 3.4. Beide Resultate sind **absolut**, also ohne Relativierung zu einer Hintergrundlogik. Auf der Automaten-Seite: floats korrespondieren zu FCMPAs, reelle Zahlen zu CMPAs. Das sind schöne, saubere Charakterisierungen.

Relativ zu MSO: Hier passiert das Interessante. GNN[F] bleibt äquivalent zu GMSC. GNN[R] ist **ebenfalls** äquivalent zu GMSC — mit Ausrufezeichen, weil das absolut nicht gilt. Und beide kollabieren auf der Automaten-Seite zu FCMPAs.

Die zwei Kernaussagen zum Mitnehmen:

- **Absolut:** GNN[F] ist strikt schwächer als GNN[R] — reelle Zahlen können undecidierbare Eigenschaften ausdrücken.
- **Relativ zu MSO:** GNN[F] und GNN[R] sind **äquivalent** — Theorie und Praxis konvergieren.

FOLIE: Conclusion & Open Questions

 ca. 2:30 min

Hinweis: Abschluss — klarer Bogen vom Anfang zum Ende. Offene Fragen kann Thomas übernehmen oder aufteilen.

Was haben wir in diesem Vortrag gezeigt?

Wir haben exakte logische Charakterisierungen für Recurrent GNNs gegeben: GNN[F] entspricht GMSC, GNN[R] entspricht ω -GML. Beide Resultate sind absolut — keine Relativierung zu einer Hintergrundlogik nötig. Und der Unterschied zwischen reellen Zahlen und Floats **verschwindet** für alle MSO-definierbaren Eigenschaften — also für alle Eigenschaften, die in der Praxis eine Rolle spielen.

Das ist das zentrale Ergebnis des Papiers: Für praktische Zwecke verlieren Floats **nichts** gegenüber reellen Zahlen. Wenn ein GNN eine praxisrelevante Eigenschaft nicht lernt, ist Float-Präzision nicht schuld.

Natürlich bleiben offene Fragen. Erstens: **Terminierung** — wie lernt ein recurrent GNN, **wann** es aufhören soll? Das ist im Papier nicht adressiert. Zweitens: **Global Readout** — wie ändert sich die Charakterisierung mit einem globalen Pooling-Layer? Drittens: **Entscheidbarkeit** — ist Ausdrückbarkeit in GMSC entscheidbar? Und viertens: **Attention-Architekturen** — wo passen GAT und Transformer in diesen Rahmen?

Diese Fragen zeigen, dass das Papier einen wichtigen Grundstein legt — aber der Bereich ist noch offen. Vielen Dank für eure Aufmerksamkeit. Fragen?

Hinweis: Bei Fragen zu Section 4 übernehmen. Bei Fragen zu Section 3 an Thomas weitergeben.

Timing-Übersicht (Ziel: 30 min)

Folie	Ziel	Puffer
GML	2:00	±30s
GMSC	1:30	±30s
ω -GML + Überleitung	1:30	±30s
→ <i>Thomas Section 3</i>	—	—
Was ist MSO?	3:00	±45s
MSO — Was es ausdrücken kann	2:00	±30s
Theorem 4.1 + Beweiszutaten	7:00	±1:00
MSO Collapse Theorem	5:00	±1:00
Warum der Unterschied verschwindet	3:00	±30s
Results at a Glance (Fazit)	2:30	±30s
Conclusion & Open Questions	2:30	±30s
Gesamt Kevin	30:00	±4:00

Wenn zu schnell: Bei den Beispielen (GMSC-Erreichbarkeit, MSO-Bipartitheit) mehr ausholen. Bei Theorem 4.1 die PTA-Konstruktion langsamer erklären und die Intuition wiederholen.

Wenn zu langsam: MSO-Beispiel-Folie kürzen (nur zwei Beispiele nennen, nicht alle). Bei Theorem 4.1 die zweite Beweiszutat (k-Präfix-Dekorationen) nur als „technisches Lemma„ nennen, ohne Details.

Annotationsverzeichnis

Alle Abkürzungen, Variablen und griechischen Buchstaben aus dem Skript

1 – Abkürzungen

- GNN** **Graph Neural Network** – neuronales Netz, das direkt auf Graphen operiert und Knotenmerkmale durch iteratives Nachrichtenaustauschen aktualisiert
- GNN[R]** GNN mit **reellen Zahlen** als Merkmalsvektoren – theoretisches Modell, unbeschränkte Präzision
- GNN[F]** GNN mit **Fließkommazahlen** (Floats) – praktisches Modell, beschränkte Präzision und Zählkapazität
- RGNN** **Recurrent GNN** – GNN ohne feste Rundenzahl; läuft bis ein Knoten einen akzeptierenden Zustand erreicht
- GML** **Graded Modal Logic** – modale Logik mit Zähloperatoren $\Diamond_{\geq k}$; charakterisiert GNNs mit fixer Rundenzahl
- GMSC** **Graded Modal Substitution Calculus** – Erweiterung von GML mit rekursiven Regeln; äquivalent zu GNN[F] (absolut)
- ω -GML** **omega-Graded Modal Logic** – Erweiterung von GML mit abzählbar unendlichen Disjunktionen; äquivalent zu GNN[R] (absolut)
- MSO** **Monadic Second-Order Logic** (Monadische Zweistufenlogik) – Erweiterung von FO mit Mengenquantifikation ($\exists X, \forall X$); erfasst praktisch alle relevanten Grapheigenschaften
- FO** **First-Order Logic** (Erstlogik) – Quantifikation über einzelne Elemente ($\exists x, \forall x$); schwächer als MSO
- PTA** **Parity Tree Automaton** (Paritätsbaumautomat) – Baumautomat, der MSO auf Bäumen charakterisiert; zentrales Werkzeug im Beweis von Thm. 4.1
- CMPA** **Counting Message-Passing Automaton** – diskretes Gegenstück zu GNN[R]; abzählbar unendliche Zustandsmenge Q
- FCMPA** **Finite CMPA** – diskretes Gegenstück zu GNN[F]; endliche Zustandsmenge Q
- AGG** **Aggregation** – Funktion, die Merkmale aller Nachbarn eines Knotens zu einem einzigen Vektor zusammenfasst (z. B. Summe, Maximum)
- COM** **Combine** – Funktion, die den eigenen Zustand eines Knotens mit dem Aggregationsergebnis kombiniert (z. B. MLP, GRU)
- GAT** **Graph Attention Network** – GNN-Variante mit lernbaren Aufmerksamkeitsgewichten auf Kanten
- MLP** **Multilayer Perceptron** – klassisches vollverbundenes neuronales Netz; häufig als COM-Funktion eingesetzt

2 – Variablen und Symbole

- G Graph – ein geordnetes Paar (V, E) aus Knotenmenge und Kantenmenge
- V Knotenmenge (**vertices**) des Graphen G
- E Kantenmenge (**edges**) des Graphen G ; $E \subseteq V \times V$
- v, u, w Einzelne Knoten im Graphen; v ist meist der Fokusknoten, u ein Nachbar
- $\mathcal{N}(v)$ Nachbarschaft von v – Menge aller Knoten u mit $(u, v) \in E$
- $\lambda(v)$ Label-Funktion – ordnet Knoten v eine Menge von atomaren Eigenschaften zu ($\lambda : V \rightarrow \mathcal{P}(\Pi)$)
- Π Menge der atomaren Eigenschaften (Propositionen), z. B. Label-Alphabet
- $\mathbf{x}_v^{(t)}$ Merkmals- bzw. Zustandsvektor von Knoten v nach Runde t
- t Aktuelle Runde (Zeitschritt) der GNN-Berechnung; $t \in \mathbb{N}_0$
- t^* Akzeptierende Runde – das erste t , für das $\mathbf{x}_v^{(t)} \in F$ gilt
- N Feste Rundenzahl bei standard GNNs – muss vorab gewählt werden
- k Zählschwelle in $\diamond_{\geq k}$ (GML) **oder** Tiefe einer k -Präfix-Dekoration (Thm. 4.1) – je nach Kontext
- n Entfaltungstiefe in GMSC – X^n ist die n -te Entfaltung des Programms
- d Dimension des Merkmalsvektors; Zustandsraum ist $\mathbb{R}^d$ bzw. $\mathbb{F}^d$
- $\mathcal{P}$ Grapheigenschaft (**property**) – eine Menge von Paaren (G, v) ; das, was ein GNN oder eine Formel ausdrückt
- Λ GMSC-Programm – besteht aus Basisregel und Iterationsregel
- X Programmvariable in einem GMSC-Programm; X^n ist die n -te Entfaltung
- X^0 Basisfall des GMSC-Programms – die Startformel φ
- $\mathcal{A}$ Paritätsbaumautomat (PTA) – akzeptiert Bäume, die eine MSO-Eigenschaft erfüllen
- Q Zustandsmenge eines Automaten (CMPA oder PTA)
- q_0 Startzustand des Automaten
- F Menge der akzeptierenden Zustände – $\mathbf{x}_v^{(t)} \in F$ bedeutet: Knoten v akzeptiert
- δ Übergangsfunktion – berechnet neuen Zustand aus aktuellem Zustand und Nachbarmultiset
- π Initialisierungsfunktion – berechnet Startzustand aus dem Label-Set des Knotens
- μ k -Präfix-Dekoration – Abbildung von Knoten der Tiefe k auf Mengen von PTA-Zuständen; endliches Akzeptanzzertifikat
- $U(G, w)$ Baumentfaltung (**tree unraveling**) von Graph G am Knoten w – unendlicher Baum aller Pfade von w aus
- $L(\mathcal{A})$ Sprache des Automaten $\mathcal{A}$ – Menge aller Bäume, die $\mathcal{A}$ akzeptiert
- $U \subseteq \mathbb{N}$ Beliebige Menge natürlicher Zahlen – GNN[$\mathbb{R}$] kann für **jedes** U prüfen, ob $|\mathcal{N}(v)| \in U$

- $M|_k$ Multiset M auf maximal k Kopien jedes Elements beschränkt — relevant für Float-Zählschranke (Prop. 2.3)
- $\mathbb{R}$ Reelle Zahlen — Zahlenbereich für $\text{GNN}[\mathbb{R}]$
- $\mathbb{F}$ Fließkommazahlen (Floats) — Zahlenbereich für $\text{GNN}[\mathbb{F}]$; endliche Darstellung, beschränkte Zählkapazität
- $\mathbb{N}$ Natürliche Zahlen $\{0, 1, 2, \dots\}$

3 — Griechische Buchstaben

- φ (**phi**) Formel — allgemein für eine logische Formel, insbesondere der Basisfall $X(\theta) :- \varphi$ in GMSC
- ψ (**psi**) Formel — Iterationsregel $X :- \psi$ in GMSC; darf X rekursiv enthalten
- δ (**delta**) Übergangsfunktion eines GNNs oder Automaten: $\delta(x, y) = \text{COM}(x, \text{AGG}(y))$
- π (**pi**) Initialisierungsfunktion: $\pi : \mathcal{P}(\Pi) \rightarrow \mathbb{R}^d$ — bildet Label-Mengen auf Startvektoren ab
- μ (**mu**) k -Präfix-Dekoration: Abbildung $\mu : V_k \rightarrow \mathcal{P}(Q)$ — weist Knoten in Tiefe k PTA-Zustandsmengen zu
- Λ (**Lambda**) GMSC-Programm, bestehend aus Basisregel und Iterationsregel
- ω (**omega**) Unendlichkeitssymbol — in ω -GML bezeichnet es die Erweiterung um abzählbar unendliche Disjunktionen
- $\diamond$ (**lozenge**) Modaloperator (**Diamant**) — $\diamond_{\geq k} \varphi$ bedeutet „mindestens k Nachfolger erfüllen φ “
- $\top$ (**top**) Tautologie — immer wahre Formel; Grundelement der GML-Syntax
- Π (**Pi**) Alphabet atomarer Eigenschaften — Menge der möglichen Labels auf Knoten
- Ψ (**Psi**) Abzählbare Menge von GML-Formeln — Grundlage einer unendlichen Disjunktion in ω -GML
- $\cap$ (**Schnitt**) Mengenschnitt — $\text{MSO} \cap \omega\text{-GML}$ ist die Menge aller Eigenschaften, die gleichzeitig in MSO und ω -GML liegen; Voraussetzung für Thm. 4.1